Hands-On Guide to LoRa, LoRaWAN, and IoT Data Pipelines with Pycom LoPy4

IntERLab

Abstract This hands-on session guides participants through building a LoRaWAN IoT system from end to end. Part 1 covers configuring Pycom LoPy4 modules for direct LoRa communication and LoRaWAN integration via The Things Network (TTN). Part 2 extends this into a complete data pipeline: capturing sensor data from TTN via MQTT, storing it in SQLite, and visualizing it with Grafana. By the end, participants will have a working system that collects, stores, and displays sensor data in real time.

Contents

1 Introduction **3**
1.1 Objective . . . 3
1.2 Prerequisites . . . 3
1.3 Schedule . . . 3
1.4 System Architecture . . . 4
1.4.1 Part 1A: Raw LoRa (Device-to-Device) . . . 4
1.4.2 Part 1B: LoRaWAN via The Things Network . . . 4
1.4.3 Part 2: Complete IoT Data Pipeline . . . 4
1.5 Required Materials . . . 4
1.6 Source Code . . . 6

Part 1: LoRa & LoRaWAN Basics **7**

2 Getting Started with the LoPy4 **7**
2.1 Connecting Thonny to the LoPy Device . . . 7
2.2 Verifying the Connection . . . 9
2.3 MicroPython Boot Sequence . . . 10
2.4 Running and Uploading Code with Thonny . . . 10
2.5 Troubleshooting . . . 11

3 Part 1A: Raw LoRa Communication **12**
3.1 What is LoRa? . . . 12
3.2 Project Setup . . . 12
3.3 Transmitter Code . . . 13
3.4 Receiver Code . . . 14
3.5 Understanding the Output: RSSI and SNR . . . 15
3.6 LoRa Parameters Reference . . . 15

4 Part 1B: LoRaWAN and The Things Network **17**
4.1 LoRa vs. LoRaWAN . . . 17
4.2 Setting Up The Things Network (TTN) . . . 17
4.2.1 Creating an Application . . . 17
4.2.2 Registering Your Device . . . 17
4.3 Sending Data via LoRaWAN (OTAA) . . . 18
4.4 Receiving Data via MQTT . . . 20
4.4.1 Generating an MQTT API Key . . . 20
4.4.2 Writing the MQTT Client . . . 21

Part 2: IoT Data Pipeline and Visualization **24**

5 Building the Data Pipeline **24**
5.1 Checkpoint: Before You Begin . . . 24
5.2 Storing Data in SQLite . . . 24
5.2.1 Creating the Database . . . 24
5.2.2 Enhanced MQTT Client with Database Storage . . . 26
5.3 Visualizing Data with Grafana . . . 28
5.3.1 Starting Grafana . . . 28
5.3.2 Logging In . . . 28
5.3.3 Adding the SQLite Data Source . . . 28
5.3.4 Building the Dashboard . . . 29

6 Exercises **31**
6.1 Additional Sensor Data . . . 31
6.2 Multiple Devices . . . 31
6.3 Data Retention . . . 31

A Software Installation Guide **32**
A.1 Thonny IDE . . . 32
A.2 Grafana . . . 32
A.3 Verification Checklist . . . 32

B Firmware Update **32**

1 Introduction

1.1 Objective

This session is divided into two parts:

Part 1: LoRa & LoRaWAN Basics (2 hours 15 minutes) Participants will:

- Set up a LoPy4 module and program it using Thonny IDE.
- Send and receive messages between two LoPy devices using raw LoRa (Part 1A).
- Connect a LoPy device to IntERLab's private LoRaWAN network via TTN and receive data on a laptop using MQTT (Part 1B).

Part 2: IoT Data Pipeline and Visualization (45 minutes) Participants will:

- Store LoRaWAN sensor data in a SQLite database.
- Build a real-time Grafana dashboard to visualize sensor readings.

1.2 Prerequisites

- Basic knowledge of Python.
- A personal laptop with administrator rights.
- Software installed per Appendix A (**complete before the session**).

1.3 Schedule

Task	Duration
Part 1: LoRa & LoRaWAN Basics	
Getting started with the LoPy4 (setup and firmware)	30 min
Part 1A: Raw LoRa communication between devices	1 hr
Part 1B: LoRaWAN with TTN and MQTT	45 min
Q&A	15 min
Part 2: IoT Data Pipeline and Visualization	
SQLite database and enhanced MQTT client	20 min
Grafana dashboard	20 min
Q&A	5 min

Table 1: Estimated schedule and duration for each activity.

Total duration: approximately 3 hours.

1.4 System Architecture

This session builds three progressively more complex systems, each illustrated with a block diagram.

1.4.1 Part 1A: Raw LoRa (Device-to-Device)

In raw LoRa mode (Figure 1), two devices communicate directly without any network infrastructure. One device transmits sensor data over a LoRa radio link, and the other receives it.

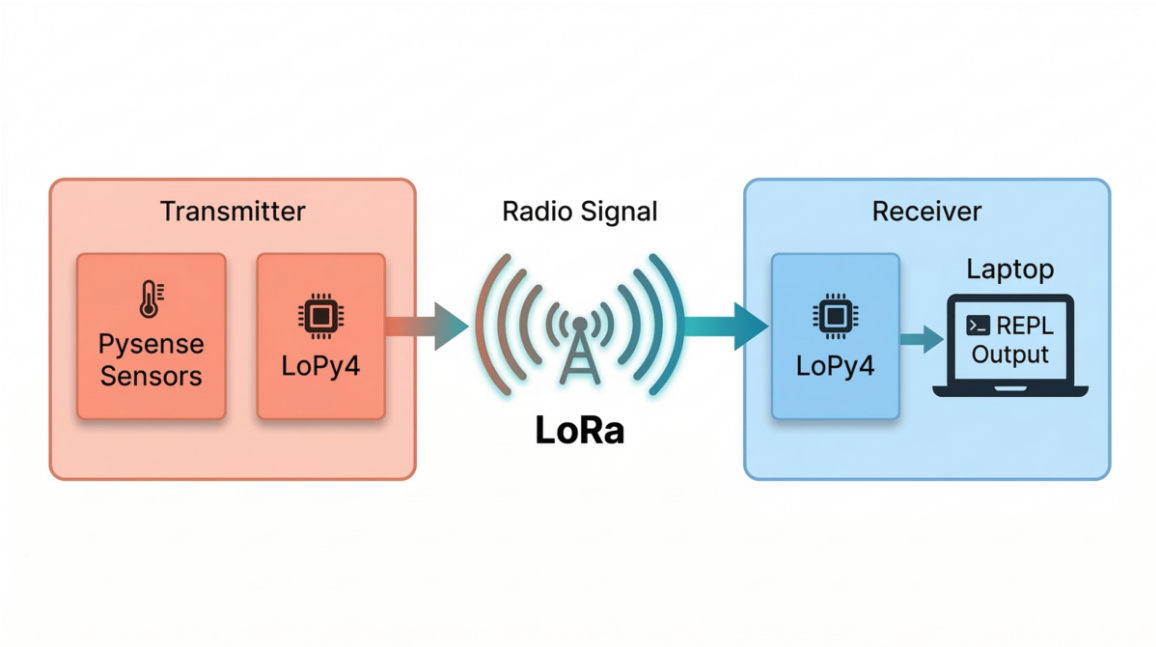


Figure 1: Part 1A: Raw LoRa device-to-device block diagram

1.4.2 Part 1B: LoRaWAN via The Things Network

In LoRaWAN mode (Figure 2), the device sends data through a gateway to a network server (TTN), which makes it available to applications via MQTT.

1.4.3 Part 2: Complete IoT Data Pipeline

Part 2 (Figure 3) extends the Part 1B pipeline by adding persistent storage (SQLite) and a visualization layer (Grafana).

1.5 Required Materials

- LoPy4 board with Pysense shield*

*Pycom, the manufacturer of the LoPy4 and Pysense, entered administration. These boards are discontinued and no longer officially supported. The existing firmware and libraries remain functional.

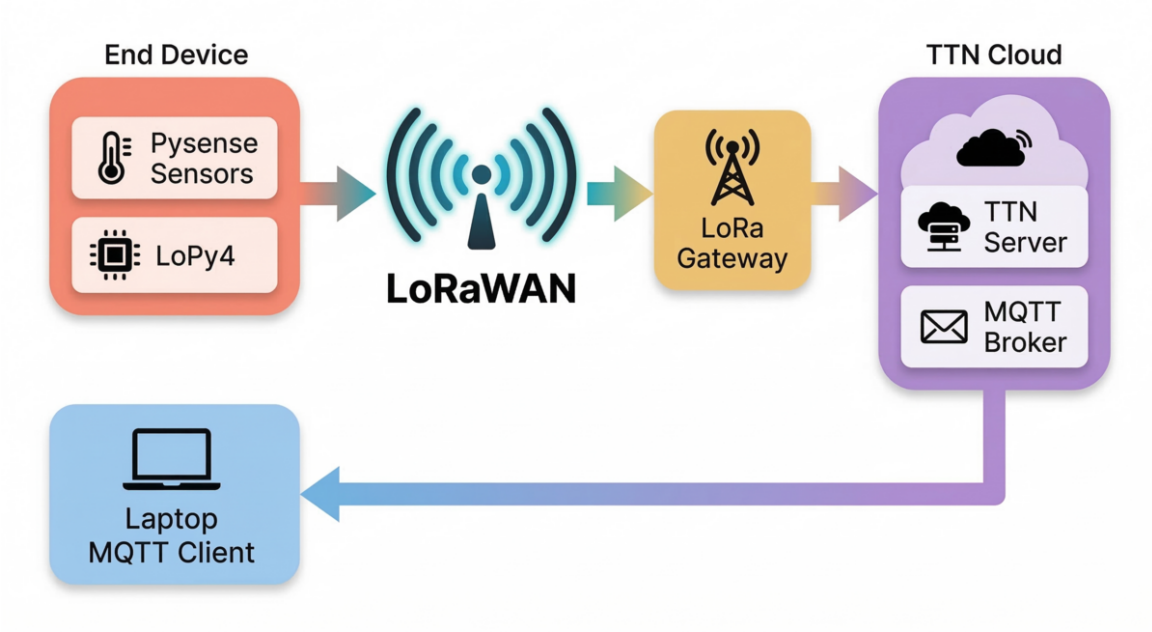


Figure 2: Part 1B: LoRaWAN network block diagram via TTN

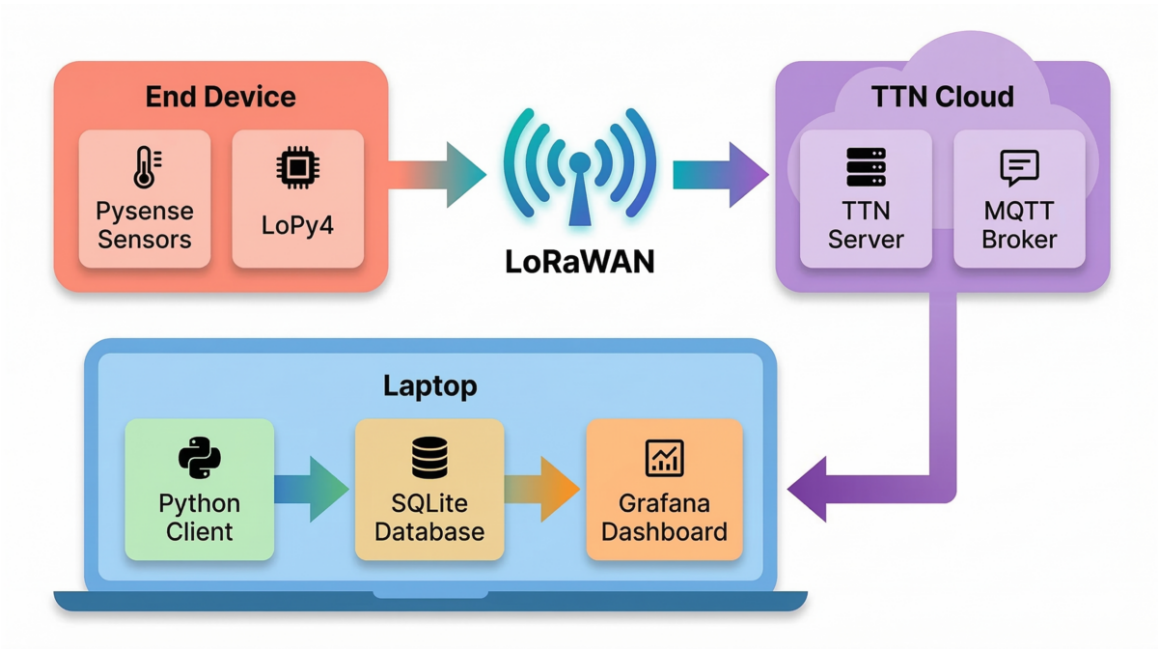


Figure 3: Part 2: Complete IoT data pipeline block diagram

- Micro-USB data cable (not power-only)
- Laptop with Python 3, Thonny IDE, and Grafana installed (see Appendix A)
- Internet connection

1.6 Source Code

All source code is available at https://github.com/rmukhia/lora-lab-session. The repository includes a `code/` directory with standalone copies of all scripts used in this tutorial for convenience:

File	Section	Runs on
`lora_transmitter.py`	Part 1A (Transmitter)	LoPy4 device
`lora_receiver.py`	Part 1A (Receiver)	LoPy4 device
`lorawan_sender.py`	Part 1B (LoRaWAN)	LoPy4 device
`mqtt_client.py`	Part 1B (MQTT)	Laptop
`create_db.py`	Part 2 (Database)	Laptop
`mqtt_to_sqlite.py`	Part 2 (Pipeline)	Laptop

Table 2: Supplementary code files

Part 1: LoRa & LoRaWAN Basics

2 Getting Started with the LoPy4

This section covers connecting to the LoPy4 device, verifying it works, and learning how to edit code directly on the device. These skills are used throughout the rest of the tutorial.

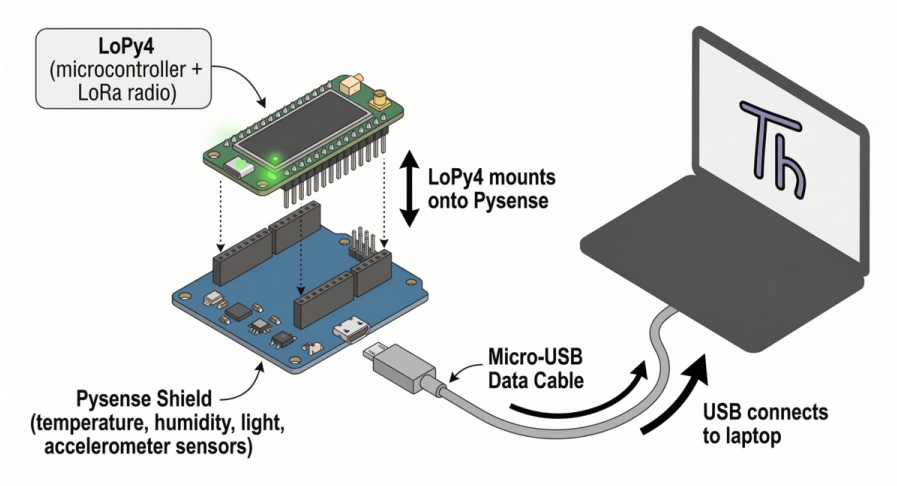


Figure 4: Hardware setup: LoPy4 mounted on Pysense shield, connected to laptop via USB

2.1 Connecting Thonny to the LoPy Device

1. Connect the LoPy4 board to your laptop via the Micro-USB cable.
2. Open Thonny and go to *Tools* → *Options* → *Interpreter* (see Figure 5).
3. Select *MicroPython* (*ESP32*) as the interpreter.
4. Select the correct Port. Look for a **Pysense** COM port (e.g., COM3). If unsure, open *Device Manager* → *Ports* (*COM & LPT*) to identify the correct port (see Figure 6).
5. Click *OK*. The MicroPython shell (REPL) should appear at the bottom of Thonny with a >>> prompt.

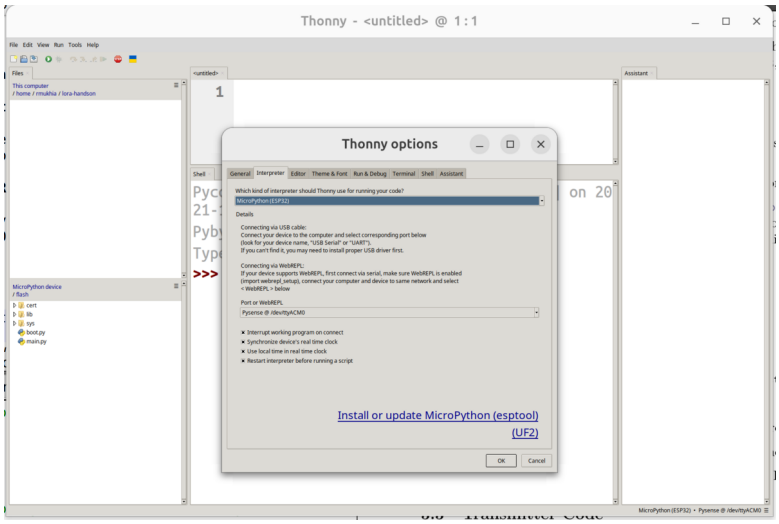


Figure 5: Thonny interpreter options window. Select *MicroPython* (*ESP32*) as the interpreter and choose the correct serial port for your LoPy4 device.

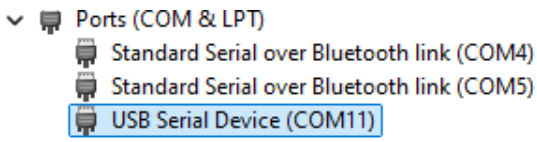


Figure 6: Windows Device Manager showing the LoPy4 COM port under *Ports* (*COM & LPT*)

For firmware update instructions, see Appendix B.

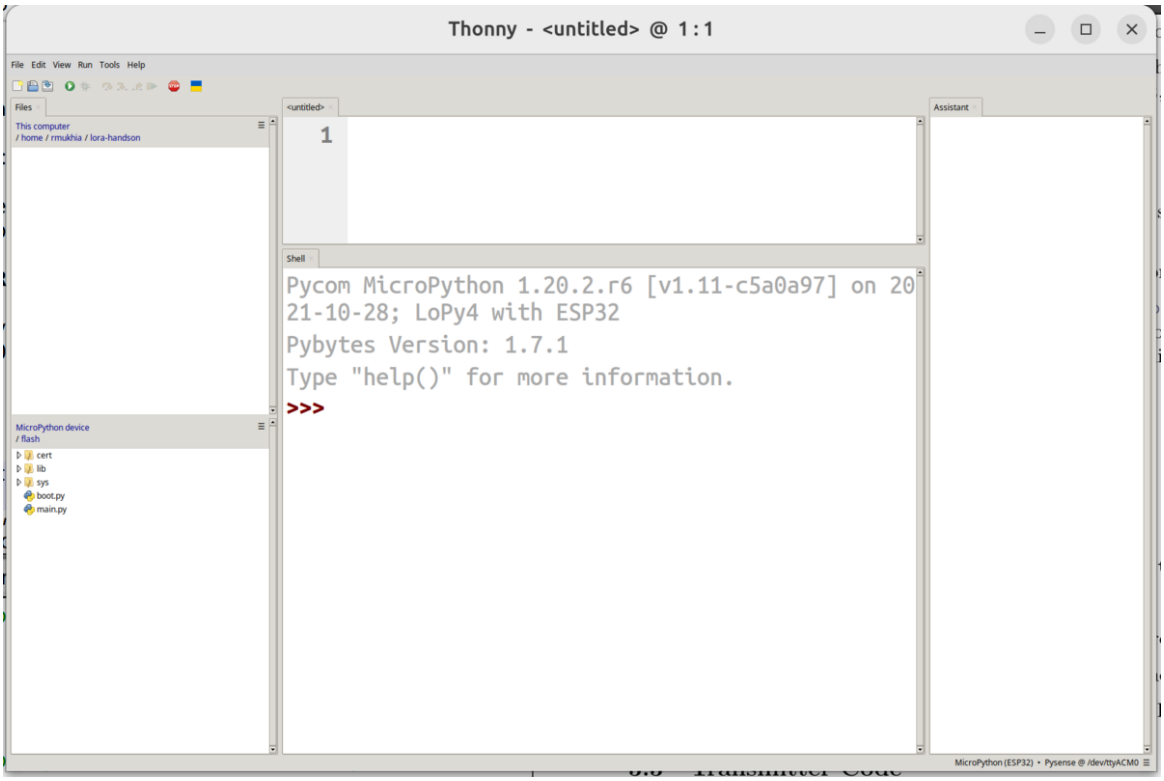


Figure 7: Thonny IDE interface. (**A**) The *file browser* shows files on your laptop (top) and on the connected MicroPython device (bottom); use it to navigate and open files directly on the device. (**B**) The *code editor* is where you write and edit scripts. (**C**) The *Shell* (*REPL*) panel at the bottom provides an interactive MicroPython prompt (>>>) for running commands directly on the device and viewing script output.

Note: If the file browser is not visible, enable it via *View* → *Files*.

2.2 Verifying the Connection

Type the following commands in the REPL to verify your device is working. Each command is typed after the >>> prompt.

```
>>> print("hello world")
hello world
>>> import pycom
>>> pycom.heartbeat(False)
>>> pycom.rgbled(0xFF0000)
>>> pycom.rgbled(0x00FF00)
>>> pycom.rgbled(0x0000FF)
```

The LED on the LoPy4 should change color with each `rgbled()` call (red, green, blue). The `heartbeat(False)` command disables the default blue LED heartbeat so you can control the LED manually.

Next, retrieve your device's unique identifiers. You will need the **DevEUI** when registering the device on TTN in Part 1B.

```
>>> import ubinascii, machine, network
>>> device_id = ubinascii.hexlify(machine.unique_id()).decode()
>>> deveui = ubinascii.hexlify(network.LoRa().mac()).decode()
>>> print("Device ID:", device_id)
Device ID: b4e62df49af5
>>> print("DevEUI:", deveui)
DevEUI: 70b3d5499b887405
```

Write down your DevEUI; you will need it in Section 4.2.2.

2.3 MicroPython Boot Sequence

A MicroPython system such as the LoPy4 has a small internal filesystem. Two files control what happens when the device starts:

- `boot.py`: Executed first, immediately after the device finishes its hardware boot. This file is typically used for low-level configuration (e.g., disabling the default heartbeat LED, setting up network interfaces). It is optional; if absent, the device simply skips it.
- `main.py`: Executed automatically after `boot.py` completes. This is where your application logic lives. In most projects, this is the file you will edit and update.

In summary, the boot sequence is: **hardware boot** → `boot.py` → `main.py`.

2.4 Running and Uploading Code with Thonny

Thonny can interact with MicroPython devices in two ways:

1. **Run without uploading:** You can write a script in Thonny's editor on your laptop and press the *Run* button (or *F5*). Thonny will send the code directly to the device and execute it immediately, without saving it to the device's filesystem. This is useful for quick testing and experimentation; the code only runs for that session and is lost when the device resets.
2. **Save to the device:** To make your code persist across reboots, you must save it to the device's filesystem:
   (a) Write or open your script in Thonny's editor.
   (b) Go to *File* → *Save as*.
   (c) Select the LoPy device (it appears as a separate location in the save dialog).
   (d) Save as `main.py` on the device. It will run automatically on every boot.

For this project, we will directly modify and save `main.py` **on the device** so that our code persists and runs automatically whenever the device is powered on.

Tip: Press *Ctrl*+*C* in the Thonny shell to stop a running script and return to the REPL. Use the *Stop* button if the device is unresponsive.

The LoPy4 device can be reset either by, pressing the reset button on the LoPy4 board, or programatically using the follow snippet on the REPL.

```
>>> import machine
>>> machine.reset()
```

2.5 Troubleshooting

- **Thonny cannot connect:** Ensure you are using a data-capable Micro-USB cable (some cables are power-only). Try a different USB port.
- **Device stuck in a loop:** Press *Ctrl*+*C* in the Thonny shell, or press the physical reset button on the LoPy4 board.
- **Need a clean slate:** Run the following in the REPL to erase all files and reset:

```
>>> import os
>>> os.fsformat("/flash")
>>> import machine
>>> machine.reset()
```

3 Part 1A: Raw LoRa Communication

In this section, you will send temperature data directly between two LoPy4 devices using raw LoRa, without any network infrastructure. One device acts as a transmitter and the other as a receiver, as shown in Figure 1.

3.1 What is LoRa?

LoRa (Long Range) is a wireless modulation technique designed for long-range, low-power communication. Key characteristics:

- Range of several kilometers (depending on environment and settings).
- Low data rate (suitable for sensor readings, not video or audio).
- Low power consumption (devices can run on batteries for months).

In *raw LoRa* mode, two devices communicate directly on the same frequency and settings, similar to a walkie-talkie. There is no network management, addressing, or security beyond what you implement yourself.

3.2 Project Setup

Before writing the transmitter and receiver code, you need to set up the Pysense sensor library on the LoPy4.

Step 1. Download the Pysense library (*pysense.zip*) from `https://github.com/pycom/pycom-libraries releases`. Extract the archive to a temporary folder on your laptop. The archive contains a *lib* folder with driver files such as `SI7006A20.py` (temperature/humidity sensor) and `pycoproc_1.py` (Pysense board interface).

Step 2. Upload the *lib* folder to the LoPy device:

1. In Thonny's file browser, locate the extracted *lib* folder on your laptop.
2. Right-click it and select *Upload to /* (or *Upload to...* and choose the device root).
3. Verify the upload by checking that *lib/* appears on the LoPy device in the file browser.

After uploading, the device filesystem should look like Figure 8.

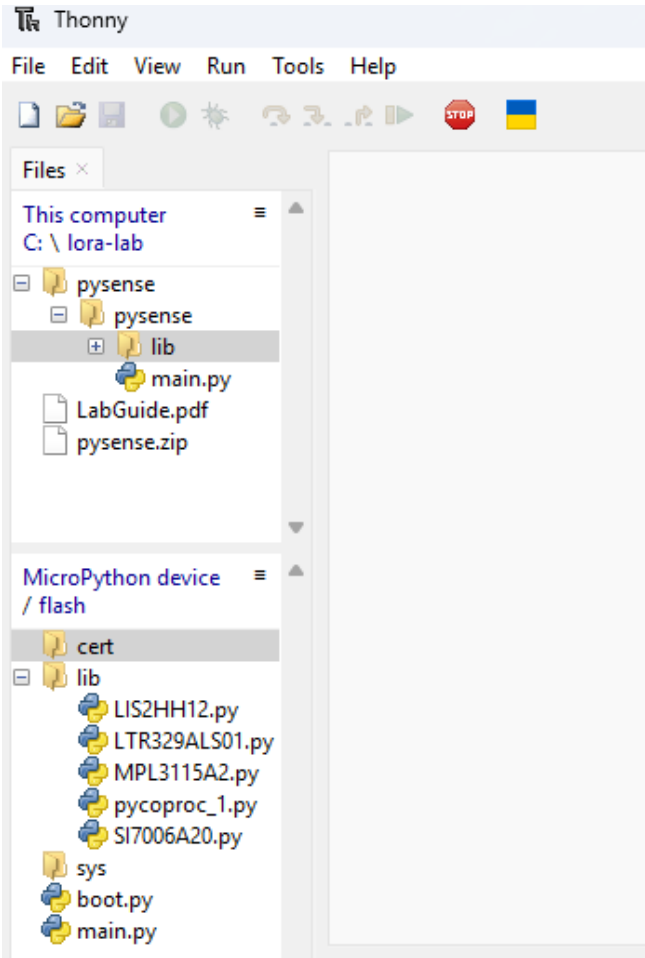


Figure 8: Upload Pysense library to the LoPy4's flash memory.

3.3 Transmitter Code

This script reads the temperature from the Pysense shield's SI7006A20 sensor and transmits it over LoRa along with the device ID. It sends 10 packets at 5-second intervals.

Step 1. In Thonny, open `main.py` on the LoPy device and replace its contents with the following code:

```
import time
from pycoproc_1 import Pycoproc
import machine
from network import LoRa
import socket
import ubinascii
from SI7006A20 import SI7006A20

py = Pycoproc(Pycoproc.PYSENSE)
si = SI7006A20(py)
```

```
def get_temp():
    """Read temperature from the SI7006A20 sensor."""
    temperature = si.temperature()
    print("Temperature: " + str(temperature) + " C")
    return temperature

# Initialize LoRa in raw LORA mode for the AS923 frequency region
lora = LoRa(mode=LoRa.LORA, region=LoRa.AS923)

# Create a raw LoRa socket
s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)
s.setblocking(False)

# Get the unique device ID
device_id = ubinascii.hexlify(machine.unique_id()).decode()

# Send 10 temperature readings
i = 0
while i < 10:
    temp = get_temp()
    packet = "{}:{}".format(device_id, temp)
    s.send(packet)
    print("Sending {}".format(packet))
    i += 1
    time.sleep(5)
```

Step 2. Save the file (*Ctrl+S*). Since you are editing `main.py` directly on the **transmitter** LoPy device, the changes are saved immediately. Reset the device or Run to execute the updated script.

Supplementary file: `code/lora_transmitter.py`

3.4 Receiver Code

This script listens for LoRa packets and prints each received message along with signal quality metrics.

Step 1. On a **separate** LoPy device (the receiver), open `main.py` in Thonny and replace its contents with the following code. The receiver does not use the Pysense sensor library, so there is no need to upload the *lib* folder to this device.

```
from network import LoRa
import socket
import time

# Initialize LoRa in raw LORA mode (must match transmitter settings)
lora = LoRa(mode=LoRa.LORA, region=LoRa.AS923)

# Create a raw LoRa socket
```

```
s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)
s.setblocking(False)

# Listen continuously for incoming packets
while True:
    rx = s.recv(64)
    if len(rx) > 0:
        print("Received message:", rx.decode(),
            "| RSSI:", lora.stats().rssi, "| SNR:", lora.stats().snr)
    time.sleep(1)
```

Step 2. Save the file (*Ctrl*+*S*). The code is saved directly to the receiver LoPy device.

Supplementary file: `code/lora_receiver.py`

Step 3. Run or reset the devices. The transmitter sends packets, and the receiver prints them in the Thonny REPL.

3.5 Understanding the Output: RSSI and SNR

The receiver prints two signal quality metrics with each packet:

- **RSSI (Received Signal Strength Indicator):** Measured in dBm. Typical values range from -30 dBm (very strong) to -120 dBm (very weak). Values above -80 dBm generally indicate a good connection.
- **SNR (Signal-to-Noise Ratio):** Measured in dB. Positive values mean the signal is stronger than the noise. LoRa can decode signals with SNR as low as -20 dB, which is one reason it achieves such long range.

Try moving the transmitter further away from the receiver and observe how RSSI and SNR change.

3.6 LoRa Parameters Reference

The transmitter and receiver must use the same LoRa settings to communicate. The following parameters can be configured when initializing LoRa on the LoPy:

Parameter	Options
Region	`LoRa.AS923`, `LoRa.AU915`, `LoRa.US915`, `LoRa.CN470`, `LoRa.IN865`
Spreading Factor	7 to 12 (higher = longer range but slower)
Bandwidth	`LoRa.BW_125KHZ`, `LoRa.BW_250KHZ`, `LoRa.BW_500KHZ`
Coding Rate	`LoRa.CODING_4_5` through `LoRa.CODING_4_8`
Preamble Length	Default: 8 symbols
Power Mode	`LoRa.ALWAYS_ON`, `LoRa.TX_ONLY`, `LoRa.SLEEP`

Table 3: Configurable LoRa parameters

The **region** setting determines the legal frequency band. This tutorial uses `LoRa.AS923` (923 MHz, Asia-Pacific). Always set the region to match local radio regulations.

4 Part 1B: LoRaWAN and The Things Network

In this section, your LoPy device will join a LoRaWAN network and transmit data that you receive on your laptop via MQTT, as shown in Figure 2.

4.1 LoRa vs. LoRaWAN

LoRa is the physical radio layer: the modulation technique you used in Part 1A for direct device-to-device communication.

LoRaWAN is a network protocol built on top of LoRa. It adds:

- **Gateways** that relay messages between end devices and a network server.
- **Device authentication and encryption** (so devices must join the network before transmitting).
- **Network management** (the server handles addressing, data rates, and routing).

Think of it this way: LoRa is like a radio transmitter, while LoRaWAN is like a cellular network built using LoRa radios.

4.2 Setting Up The Things Network (TTN)

IntERLab maintains a private TTN instance at *ttn.hazemon.in.th*. You will create an application and register your device on this server.

4.2.1 Creating an Application

1. Log in to *ttn.hazemon.in.th* using the credentials provided.
2. Navigate to the *Applications* tab.
3. Click *Add Application*.
4. Enter an *Application ID* (use something unique and memorable, such as your student ID). Click *Create Application*.

4.2.2 Registering Your Device

Each device must be registered under an application before it can join the network.

1. In your application, click *Add end device*, then select the *manual* registration option.
2. Fill in the following fields:
   - **LoRaWAN parameters:** The Frequency plan, LoRaWAN version, Regional Parameters version should be as shown in Figure 9.
   - **DevEUI:** The value you recorded in Section 2.2.
   - **AppEUI:** Fill with zeros (`0000000000000000`).

- **AppKey:** Click *Generate* to create a random key. **Copy this key**; you will need it in the next step.
- **End device ID:** A unique name for this device.

3. Click *Register end device*.

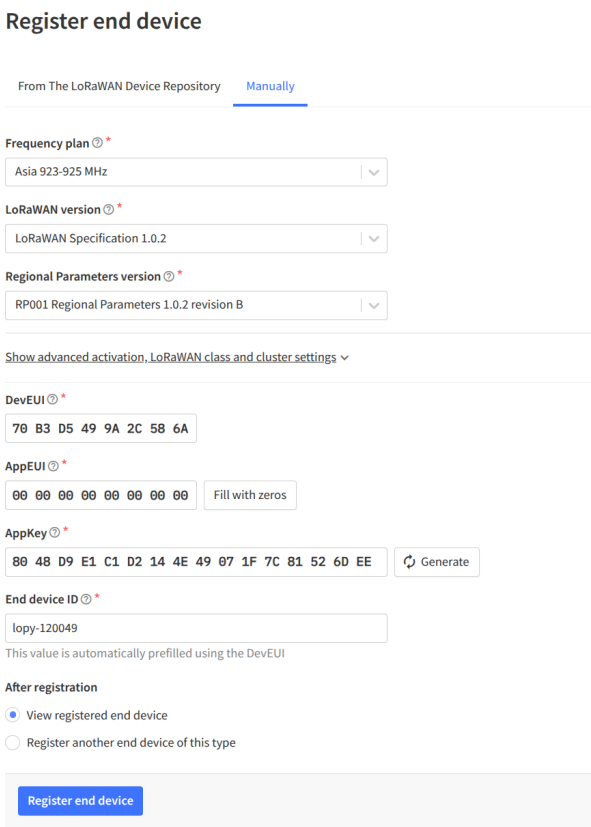


Figure 9: Registering an end device on TTN

The device is registered as a **Class A** device (the most common type: the device initiates all communication and only listens for downlinks briefly after each uplink). More on device classes: https://www.thethingsnetwork.org/docs/lorawan/classes/.

4.3 Sending Data via LoRaWAN (OTAA)

The device joins the TTN network using **OTAA (Over-The-Air Activation)**: it sends a join request containing its DevEUI and AppKey, and the network responds with session keys if authentication succeeds. This is more secure than the alternative (ABP), where keys are hard-coded. More details: https://www.thethingsindustries.com/docs/hardware/devices/concepts/abp-vs-otaa/.

Step 1. In Thonny, open `main.py` on the LoPy device and replace its contents with the following code. Replace `YOUR_APP_KEY_HERE` with the AppKey generated in Section 4.2.2 (a

32-character hex string).

```
import time
from pycoproc_1 import Pycoproc
import machine
from network import LoRa
import socket
import ubinascii
from SI7006A20 import SI7006A20

py = Pycoproc(Pycoproc.PYSENSE)
si = SI7006A20(py)

def get_temp():
    """Read temperature from the SI7006A20 sensor."""
    temperature = si.temperature()
    print("Temperature: " + str(temperature) + " C")
    return temperature

# Initialize LoRa in LORAWAN mode
lora = LoRa(mode=LoRa.LORAWAN, region=LoRa.AS923)

# OTAA authentication credentials from TTN
app_eui = ubinascii.unhexlify("0000000000000000")
app_key = ubinascii.unhexlify("YOUR_APP_KEY_HERE")  # Replace with your AppKey

# Join the network (OTAA)
lora.join(activation=LoRa.OTAA, auth=(app_eui, app_key), timeout=0)

# Wait for the join to complete
while not lora.has_joined():
    time.sleep(2.5)
    print("Not yet joined...")

print("Joined the network!")

# Create a LoRa socket and configure it
s = socket.socket(socket.AF_LORA, socket.SOCK_RAW)
s.setsockopt(socket.SOL_LORA, socket.SO_DR, 5)  # Set data rate
s.setblocking(True)  # Wait for TX to complete before returning

# Send 100 temperature readings
i = 0
while i < 100:
    temp = get_temp()
    packet = "Temperature: {}".format(temp)
    s.send(packet)
    print("Sending {}".format(packet))
    i += 1
```

```
time.sleep(5)
```

Step 2. Save the file (*Ctrl+S*) to store the changes directly on the LoPy device. Reset the device to run the updated script.

Supplementary file: `code/lorawan_sender.py`

Step 3. Observe the REPL output. The device will print "Not yet joined..." until it successfully authenticates, then begin transmitting temperature readings.

Step 4. In TTN, open your application and go to the *Live data* tab to confirm that uplink messages are arriving.

Note: The payload format here (`"Temperature: 28.7"`) differs from Part 1A, which used `"device_id:temp"`. The Part 2 data pipeline expects the `"Temperature: ..."` format used in this section. If you reuse the Part 1A transmitter code without updating the payload format, the `extract_temperature()` function in Part 2 will not be able to parse the data.

Note: Sending the temperature as a human-readable string is convenient for debugging, but wasteful for LoRaWAN where payload sizes are severely limited. In production systems, you would encode sensor values as compact binary data. We use strings here for tutorial clarity; see the Exercises section for a more efficient approach.

4.4 Receiving Data via MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe messaging protocol commonly used in IoT. In this model:

- A **broker** (the TTN server) receives messages from publishers and forwards them to subscribers.
- **Topics** are hierarchical strings (like `v3/my-app/devices/my-device/up`) that identify message streams.
- A **subscriber** connects to the broker and requests messages matching a topic.

TTN acts as an MQTT broker: when your LoPy device sends an uplink, TTN publishes it to an MQTT topic. You will write a Python script on your laptop that subscribes to this topic and prints the data.

4.4.1 Generating an MQTT API Key

1. In TTN, go to your application → *Integrations* → *MQTT*.
2. Click *Generate new API key*.
3. **Copy and save the API key immediately**; it is shown only once.

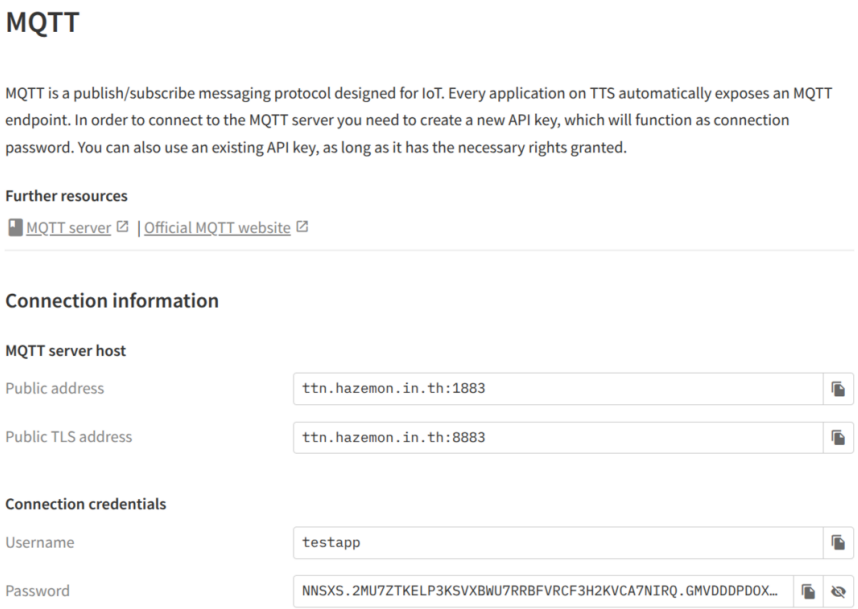


Figure 10: TTN MQTT credentials page

4.4.2 Writing the MQTT Client

The MQTT client script runs on your **laptop**, not on the LoPy device. From this point onward, all client-side Python scripts (`mqtt_client.py`, `create_db.py`, `mqtt_to_sqlite.py`) are written and run using your laptop's local Python 3 interpreter in Thonny.

Tip: It is convenient to keep **two Thonny instances** open side by side, one set to *MicroPython (ESP32)* for the LoPy4 device code, and another set to *Python 3* for the laptop client scripts. This way you can monitor device output and run client code without switching interpreters back and forth.

To enable multiple Thonny instances *Allow only single Thonny instance* should be disabled in *Tools* → *Options* → *General*.

Switching the Thonny interpreter: Go to *Tools* → *Options* → *Interpreter* and select the default *Python 3* interpreter (instead of *MicroPython (ESP32)*). Keep this setting for all laptop scripts in Part 1B and Part 2.

Step 1. Create a new project directory for the Python MQTT client.

Step 2. In Thonny, goto *Tools* → *Manage Packages* and install paho-mqtt package for MQTT support.

Step 3. Create a new file named `mqtt_client.py` in your project folder with the following code. Replace the three placeholder values with your TTN credentials:

```
import paho.mqtt.client as mqtt
```

```python
import json
import base64

# TTN credentials (replace these with your values)
USERNAME = "YOUR_TTN_APP_ID"        # Your TTN Application ID
PASSWORD = "YOUR_TTN_API_KEY"       # The API key generated above
DEVICE_ID = "YOUR_DEVICE_ID"       # The end device ID from TTN

def on_connect(client, userdata, flags, rc, properties=None):
    """Subscribe once connected (ensures re-subscription on reconnect)."""
    print(f"Connected with result code {rc}")
    client.subscribe(f"v3/{USERNAME}/devices/{DEVICE_ID}/up")

def on_message(client, userdata, msg):
    """Called each time a message arrives from TTN."""
    data = json.loads(msg.payload.decode())

    # TTN encodes the device payload in base64
    payload_base64 = data["uplink_message"]["frm_payload"]
    payload = base64.b64decode(payload_base64).decode()

    print(f"Received message: {payload}")

# MQTT client setup
client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message

client.username_pw_set(USERNAME, PASSWORD)

# Connect to the TTN MQTT broker
BROKER = "ttn.hazemon.in.th"
PORT = 1883
client.connect(BROKER, PORT)

# Listen forever (press Ctrl+C to stop)
client.loop_forever()
```

Note: TTN encodes device payloads in Base64 within its JSON messages. Base64 is a text encoding that represents arbitrary binary data using only printable ASCII characters, ensuring the payload can be safely embedded in JSON without corruption. The `base64.b64decode()` call converts it back to the original bytes.

Step 3. Run the script by pressing the *Run* button (or *F5*) in Thonny.

Supplementary file: `code/mqtt_client.py`

You should see temperature readings printed each time the LoPy device transmits:

```
Received message: Temperature: 28.73
```

```
Received message: Temperature: 28.81
Received message: Temperature: 28.79
```

Part 2: IoT Data Pipeline and Visualization

5 Building the Data Pipeline

In Part 1B, you received sensor data on your laptop and printed it to the terminal. In Part 2, you will extend this into a complete IoT data pipeline (Figure 3) by storing the data in a database and visualizing it with a dashboard.

5.1 Checkpoint: Before You Begin

Before proceeding with Part 2, complete the following two preparatory steps:

1. Modify the LoPy device code to transmit continuously. In Thonny, open `main.py` on the LoPy device and replace the limited loop from Part 1B:

```
# Change this:
i = 0
while i < 100:
    ...
    i += 1

# To this:
while True:
    temp = get_temp()
    packet = "Temperature: {}".format(temp)
    s.send(packet)
    print("Sending {}".format(packet))
    time.sleep(5)
```

Save the file (*Ctrl+S*) and reset the device so it keeps transmitting throughout Part 2.

2. Ensure Grafana is installed per Appendix A. You will need it in Section 5.3.

5.2 Storing Data in SQLite

SQLite is a lightweight relational database engine built into Python's standard library, requiring no separate server installation. It stores the entire database in a single file, making it ideal for local data collection.

5.2.1 Creating the Database

Step 1. Verify SQLite is available on your system. Type the following in the Thonny shell:

```
>>> import sqlite3
>>> print(sqlite3.version)
```

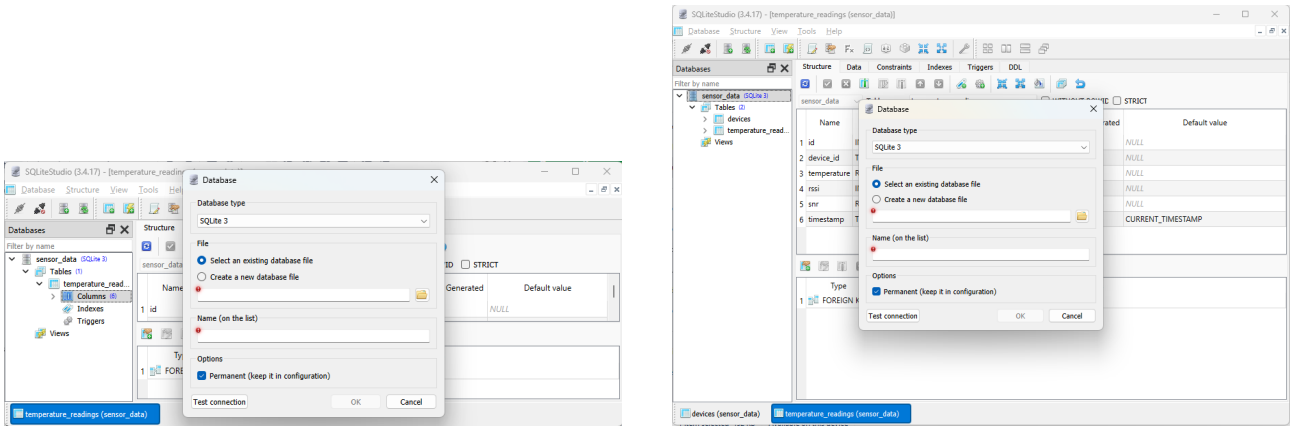
Step 2. In the same project folder as your MQTT client (from Section 4.4), create a file named `create_db.py`:

```python
import sqlite3

DB_FILE = "sensor_data.db"

conn = sqlite3.connect(DB_FILE)
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS temperature_readings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    device_id TEXT,
    temperature REAL,
    rssi INTEGER,
    snr REAL,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)
""")

conn.commit()
conn.close()

print(f"Database created successfully at {DB_FILE}")
```

Step 3. Open `create_db.py` in Thonny and run it by pressing the *Run* button (or *F5*).

Supplementary file: `code/create_db.py`

Step 4. (Optional) Verify the database was created correctly using a tool such as *SQLiteStudio* (https://github.com/pawelsalawa/sqlitestudio/releases/) or *DB Browser for SQLite* (https://sqlitebrowser.org/dl/). Open the `sensor_data.db` file and confirm the `temperature_readi` table exists.



(a) Table structure (b) Table data view

Figure 11: Database structure and data viewed in SQLiteStudio

5.2.2 Enhanced MQTT Client with Database Storage

This script is an enhanced version of the Part 1B MQTT client. Instead of just printing messages, it extracts the temperature, RSSI, and SNR from each TTN uplink and stores them in the SQLite database.

Important: Create and run all Part 2 scripts in the **same project folder** you set up in Section 4.4.

Step 1. Create a file named `mqtt_to_sqlite.py`:

```
import paho.mqtt.client as mqtt
import json
import base64
import sqlite3
import re

# Configuration (replace with your values)
DB_FILE = "sensor_data.db"
BROKER = "ttn.hazemon.in.th"
PORT = 1883
USERNAME = "YOUR_TTN_APP_ID"        # Your TTN Application ID
PASSWORD = "YOUR_TTN_API_KEY"       # Your TTN API key
DEVICE_ID = "YOUR_DEVICE_ID"       # Your TTN end device ID

def extract_temperature(payload):
    """Extract the numeric temperature from a 'Temperature: 25.3' string."""
    match = re.search(r"Temperature: ([\d.]+)", payload)
    if match:
        return float(match.group(1))
    return None

def on_connect(client, userdata, flags, rc, properties=None):
    """Subscribe to the device topic once connected."""
    print(f"Connected with result code {rc}")
    client.subscribe(f"v3/{USERNAME}/devices/{DEVICE_ID}/up")

def on_message(client, userdata, msg):
    """Process each incoming message and store it in the database."""
    try:
        data = json.loads(msg.payload.decode())

        # Decode the device payload (base64-encoded by TTN)
        payload_base64 = data["uplink_message"]["frm_payload"]
        payload = base64.b64decode(payload_base64).decode()

        # Extract signal quality metadata from the gateway
        rssi = data["uplink_message"]["rx_metadata"][0]["rssi"]
        snr = data["uplink_message"]["rx_metadata"][0]["snr"]

        temperature = extract_temperature(payload)

        if temperature is not None:
            conn = sqlite3.connect(DB_FILE)
            cursor = conn.cursor()
```

```
            cursor.execute(
                "INSERT INTO temperature_readings "
                "(device_id, temperature, rssi, snr) VALUES (?, ?, ?, ?)",
                (DEVICE_ID, temperature, rssi, snr)
            )
            conn.commit()
            conn.close()
            print(f"Stored: Temp={temperature} C, RSSI={rssi}, SNR={snr}")
        else:
            print(f"Could not parse temperature from: {payload}")

    except Exception as e:
        print(f"Error processing message: {e}")

# Set up and start the MQTT client
client = mqtt.Client(callback_api_version=mqtt.CallbackAPIVersion.VERSION2)
client.on_connect = on_connect
client.on_message = on_message
client.username_pw_set(USERNAME, PASSWORD)
client.connect(BROKER, PORT)

print(f"Listening for data from device {DEVICE_ID}...")
print("Press Ctrl+C to stop")
client.loop_forever()
```

Step 2. Replace `USERNAME`, `PASSWORD`, and `DEVICE_ID` with your TTN credentials (same values as in Part 1B).

Supplementary file: `code/mqtt_to_sqlite.py`

Step 3. Open `mqtt_to_sqlite.py` in Thonny and run it by pressing the *Run* button (or *F5*).

As the LoPy device transmits, you should see "Stored: Temp=... C, RSSI=..., SNR=..." messages confirming data is being saved.

Step 4. (Optional) Open `sensor_data.db` in SQLiteStudio or DB Browser to verify that rows are being inserted.

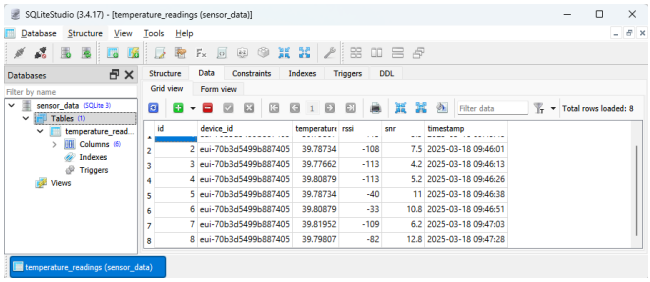


Figure 12: Temperature readings stored in the SQLite database

---

Keep `mqtt_to_sqlite.py` **running** while you set up Grafana in the next section. It needs to continue collecting data so the dashboard has something to display.

5.3 Visualizing Data with Grafana

Grafana is an open-source visualization platform that connects to data sources and displays real-time dashboards. You will connect it to your SQLite database to visualize temperature, RSSI, and SNR data.

5.3.1 Starting Grafana

Start Grafana using the method for your operating system:

- **Windows:** Launch from the Start menu, or run `grafana-server.exe` from the extracted directory.
- **macOS:** Run `./bin/grafana server` from the Grafana directory.
- **Linux:** Run `sudo systemctl start grafana-server`.

5.3.2 Logging In

1. Open `http://localhost:3000` in a web browser.
2. Log in with username `admin` and password `admin`.
3. Change the password when prompted.

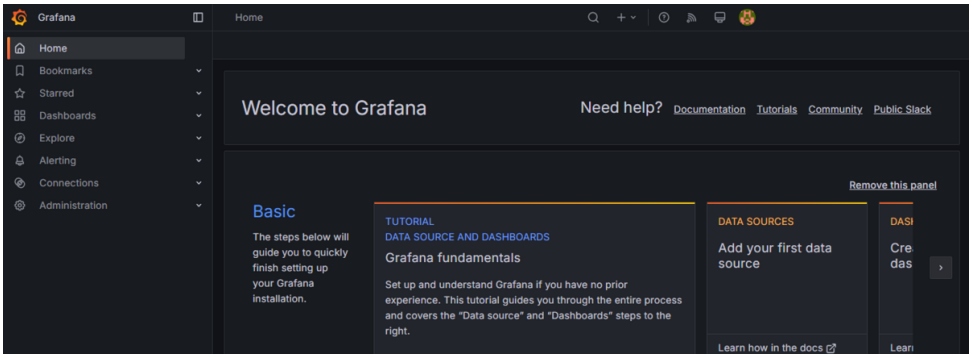


Figure 13: Grafana landing page

5.3.3 Adding the SQLite Data Source

1. Install the SQLite plugin: go to *Administration* → *Plugins and data* → *Plugins*, search for "SQLite", and click *Install*.
2. Add a data source: go to *Connections* → *Data Sources* → *Add data source*.
3. Select "SQLite" from the list.
4. Set the name to `LoRaWAN Sensor Data`.
5. Enter the full path to your `sensor_data.db` file (e.g., `C:\Users\you\project\sensor_data.db` on Windows or `/home/you/project/sensor_data.db` on Linux/macOS).

6. Click *Save & Test* to verify the connection.

5.3.4 Building the Dashboard

1. Go to *Dashboards* → *Create Dashboard* → *Add visualization*.
2. Select *LoRaWAN Sensor Data* as the data source.
3. **Temperature time series panel:** Enter the following SQL query:

   ```
   SELECT
     cast(strftime('%s', timestamp) as integer) AS time,
     temperature AS value,
     device_id AS metric
   FROM temperature_readings
   ORDER BY timestamp ASC;
   ```

   Set the visualization type to *Time series*, the title to "Temperature Readings", and the Y-axis label to "Celsius".

   Note: The `strftime('%s', timestamp)` expression returns Unix epoch time in **seconds**. The SQLite plugin for Grafana expects seconds by default. If you see "No data" or an incorrect time axis, check that the *time* column is recognized by Grafana: in the query inspector, verify the returned values are reasonable Unix timestamps. Some Grafana panel types or data source plugins may expect milliseconds instead; in that case, change the query to use `cast(strftime('%s', timestamp) as integer) * 1000 AS time`.

4. **Current temperature gauge:** Add a new panel with this query:

   ```
   SELECT temperature as value
   FROM temperature_readings
   ORDER BY timestamp DESC
   LIMIT 1
   ```

   Set the visualization type to *Gauge*, the title to "Current Temperature", min/max to 0/50, and add color thresholds at 35 (warning) and 45 (critical).

5. Save the dashboard as "LoRaWAN Sensor Dashboard".

Figure 14: Completed LoRaWAN Sensor Dashboard. The time range and refresh interval can be adjusted in the top-right corner.

Exercise: Add a panel for RSSI (signal strength) to the dashboard. Try to replicate the layout shown in Figure 14.

6 Exercises

These exercises are optional and can be attempted if time permits.

6.1 Additional Sensor Data

Extend the LoPy code to transmit humidity, pressure, and luminosity alongside temperature, and add corresponding panels to the Grafana dashboard. Note: at SF12 with the AS923 dwell time restriction (400ms), the maximum payload is only 11 bytes. How would you encode multiple sensor readings within this limit?

6.2 Multiple Devices

The current setup uses a single device. How would you extend it to handle data from multiple sensors registered under the same TTN application?

Hint: MQTT supports wildcard subscriptions (e.g., `v3/my-app/devices/+/up` matches all devices).

6.3 Data Retention

To prevent the database from growing indefinitely, write a script (`cleanup_data.py`) that deletes records older than 15 minutes. Schedule it to run periodically using cron (Linux/macOS) or Task Scheduler (Windows).

Note: In production systems, data retention typically involves archiving old data to a separate backup store rather than deleting it.

A Software Installation Guide

Complete these installations **before the session**.

A.1 Thonny IDE

Thonny is a Python IDE with built-in MicroPython support for programming microcontrollers.

1. Download from https://thonny.org/ and install for your OS with Python.
2. Launch Thonny to verify the installation.

A.2 Grafana

- **Windows:** Download from https://grafana.com/grafana/download?edition=enterprise&platform=windows. Install using the installer or extract the zip version.
- **macOS:** Download from https://grafana.com/grafana/download?edition=enterprise&platform=mac. Extract and run `./bin/grafana server`.
- **Linux:** Follow instructions at https://grafana.com/docs/grafana/latest/installation/.

A.3 Verification Checklist

Before the session, confirm:

✓ Thonny IDE installed and launches successfully

✓ Grafana installed (verify by starting it and accessing `http://localhost:3000`)

✓ Laptop has administrator rights (may be needed for USB drivers)

B Firmware Update

If the LoPy4 firmware needs to be updated, use the Pycom firmware update tool.

1. Download the tool from https://docs.pycom.io/updatefirmware/device/.
2. Connect the LoPy4 via USB. The tool usually detects the port automatically.
   - **Windows:** Check *Device Manager* → *Ports* (*COM & LPT*) for the correct port.
   - **macOS/Linux:** Run `ls /dev/tty*` to identify the port.
3. Set *Type* to *pybytes*.
4. Check *Show Advanced Settings* and skip Pybytes registration.
5. (Optional) Select *Erase during update* to clear the device filesystem.

6. Click *Update* and wait for the process to complete (see Figure 15).
7. Note the *Device ID* and *LoRa MAC* shown on the result screen.

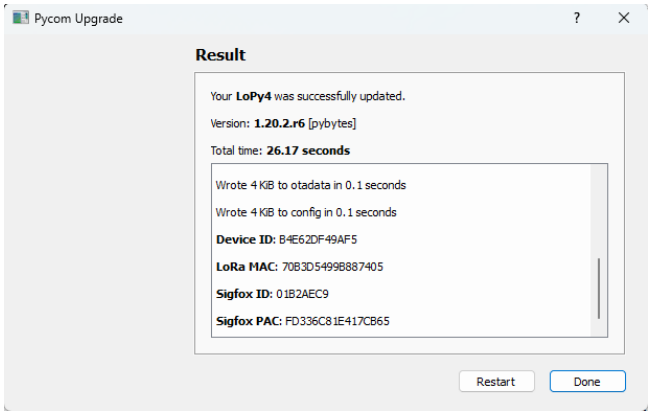


Figure 15: Firmware update completed successfully. Note the Device ID and LoRa MAC displayed.